CHƯƠNG 3: PHƯƠNG PHÁP THỰC HIỆN

3.1 Công cụ thực hiện

Nhóm sử dụng ngôn ngữ lập trình python và lập trình trên phần mềm VScode.
Truy cập mã nguồn mở của chương trình tại:
https://github.com/BAROKHUU/Calculus_II_HCMUT

3.2. Khởi tạo thư viện cần thiết
Chương trình sử dụng thư viện math, numpy, random và keras(từ tensorflow).

```
import math
import random
import numpy as np
import tkinter as tk
from PIL import Image, ImageDraw
from keras.datasets import mnist
```

3.3. Tiền xử lý dữ liệu và biểu diễn bài toán dưới dạng nhiều biến

Chương trình sử dụng bộ dữ liệu MNIST, với MNIST là bộ dữ liệu chữ viết tay gồm các số từ 0 đến 9.

```
def load_datasets():
  (x_train, y_train), (x_test, y_test) = mnist.load_data()

  #Only 0 and 1 for classification
  train_mask = (y_train == 0) | (y_train == 1)
  test_mask = (y_test ==0) | (y_test == 1)

  x_train = x_train[train_mask]
  y_train = y_train[train_mask]

  x_test = x_test[test_mask]
  y_test = y_test[test_mask]

  #normalize from [0,255] to [0,1] <=> RGB to grayscale
  x_train = x_train.astype(np.float64) / 255.0
  x_test = x_test.astype(np.float64) / 255.0

  #image 28x28 pixels into 784 dims vector
  x_train = x_train.reshape(x_train.shape[0], -1)
  x_test = x_test.reshape(x_test.shape[0], -1)
```

```
y_train = y_train.astype(np.float64)
y_test = y_test.astype(np.float64)

return x_train, y_train, x_test, y_test
```

Việc xử lý dữ liệu bao gồm các bước:

1. Lọc và chỉ lấy các dữ liệu có nhãn 0 và 1.
2. Chia dữ liệu thành 2 tập là huấn luyện và thử nghiệm (với 12665 mẫu huấn luyện và 2115 mẫu thử nghiệm, mẫu 0 và 1 xen kẽ nhau).
3. Chuẩn hóa giá trị của các điểm ảnh [0,255] thành [0,1] giúp quá trình tối ưu và ổn định hơn vì nếu độ lệch của dữ liệu quá lớn thì giá trị của tích vô hướng giữa đầu vào và trọng số có thể rất lớn, làm hàm sigmoid bão hòa nhanh, dẫn đến gradient nhỏ và quá trình học chậm hoặc kém ổn định.
4. Chuyển ảnh có kích thước 28x28 thành vector 784 chiều (Khi đó mỗi mẫu dữ liệu sẽ được biểu diễn dưới dạng $x = (x_1, x_2, \ldots, x_{784})$ và mô hình Logistic Regression sẽ sử dụng vector trọng số(weights) $w = (w_1, w_2, \ldots, w_{784})$ kèm theo độ lệch(bias) để tìm bộ tham số phù hợp cho hàm mất mát(Loss function) đạt giá trị nhỏ nhất).

3.4. Xây dựng mô hình Logistic Regression

3.4.1. Hàm Sigmoid

Logistic Regression sử dụng hàm sigmoid để biến đầu ra tuyến tính thành xác suất:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

tương đương với hàm được cài đặt trong chương trình:

```
def sigmoid(z):
  z = np.clip(z, -500, 500)
  return 1.0 / (1.0 + np.exp(-z))
```

- np.exp(-z) = e^{-z}
- np.clip(z, -500, 500) dùng để tránh tràn số khi z quá lớn hoặc quá nhỏ.

3.4.2. Mô hình dự đoán xác suất

Theo lý thuyết, với một mẫu dữ liệu x, mô hình Logistic Regression tính $z = w^T x+b$ sau đó lấy y_hat = σ(z) với y_hat là mô hình dự đoán.

```
#return predict probs of y_hat
def predict_probability(X ,weights, bias):
  z = X @ weights + bias # @ matrix multiplication
  return sigmoid(z)
```

- X @ weights là phép nhân ma trận, thực hiện đồng thời tích vô hướng giữa từng mẫu dữ liệu với vector trọng số
- + bias: cộng thêm hệ số chệch
- Sigmoid(z): biến giá trị tuyến tính thành xác suất thuộc lớp 1.

3.4.3. Dự đoán nhãn

```
#predict label 0 or 1 from probs
def predict(X, weights, bias, threshold=0.5):
  probs = predict_probability(X, weights, bias)
  return (probs >= threshold).astype(np.int32)
```

Chương trình sử dụng ngưỡng để xác định nhãn, với ngưỡng là 0.5, nếu lớn hơn hoặc bằng ngưỡng thì y_hat có nhãn là 1 còn nếu bé hơn ngưỡng thì y_hat có nhãn 0.

3.5. Hàm mất mát Binary-cross Entropy

Hàm được cài đặt trong chương trình như sau:

```
def loss_func(y_true, y_hat):
  epsilon = 1e-10

  y_hat = np.clip(y_hat, epsilon, 1-epsilon)
  #epsilon & 1-epsilon are limits of y_hat

  loss = -np.mean(y_true*np.log(y_hat) + (1.0 - y_true)*np.log(1.0-y_hat))

  return loss
```

y_hat được np.clip() giới hạn giá trị để không vượt quá epsilon và 1-epsilon.
np.mean đóng vai trò chia cho số mẫu m
np.log() là sử dụng log.

3.6. Đạo hàm nhiều biến trong hàm tính Gradient

```
def gradients_func(X, y_true, y_hat):

  m = X.shape[0]
  error = y_hat-y_true

  d_weights = (X.T @ error) / m
  d_bias = np.sum(error) / m

  return d_weights, d_bias
```

- m = X.shape[0]: số mẫu huấn luyện
- error = y_hat - y_true: vector sai số (y^−y)(\hat{y} - y)(y^−y)
- X.T @ error = X^T(y_hat−y)
- np.sum(error)/m: công thức đạo hàm theo bias.

Đây là đoạn code thể hiện rõ nhất việc áp dụng **đạo hàm nhiều biến**. Thay vì tính riêng lẻ từng đạo hàm, chương trình tận dụng đại số ma trận để tính toàn bộ vector gradient trong một phép toán.

3.7. Gradient Descent và quá trình cập nhật tham số

```
def Logistic_Regression_Training(X, y, learning_rate=0.12, epochs = 200):
  n_features = X.shape[1]

  weights = np.zeros(n_features, dtype=np.float64)
  bias = 0.0

  loss_history = []
  accuracy_history = []

  for epoch in range(epochs):
     y_hat = predict_probability(X, weights, bias)

     loss = loss_func(y, y_hat)

     d_weigths, d_bias = gradients_func(X, y, y_hat)

     weights = weights - learning_rate*d_weigths
     bias = bias - learning_rate*d_bias
```

```
    y_hat = predict(X, weights, bias)
    accuracy = accuracy_func(y, y_hat)

    loss_history.append(loss)
    accuracy_history.append(accuracy)

    if epoch == 0 or (epoch+1)%10==0:
        print(f"Epoch {epoch +1:3d}/{epoch} | "
              f"Loss: {loss:.4f} | "
              f"Train accuray: {accuracy:.4f}")

  return weights, bias, loss_history, accuracy_history
```

Ban đầu, vector trọng số w và bias b được khởi tạo bằng 0. Trong mỗi vòng lặp (epoch), mô hình thực hiện các bước sau:

1. **Tính đầu ra dự đoán**:
   Sử dụng công thức y_hat=σ(Xw+b) để tính xác suất dự đoán.
2. **Tính hàm mất mát**:
   Áp dụng hàm Binary Cross-Entropy để đo sai số giữa dự đoán và nhãn thực.
3. **Tính gradient**:
   Sử dụng đạo hàm nhiều biến để tính các đạo hàm riêng của hàm mất mát theo từng trọng số và bias:

$$\frac{\partial J}{\partial w}, \quad \frac{\partial J}{\partial b}$$

4. **Cập nhật tham số:**
   Áp dụng quy tắc Gradient Descent:

$$w := w - \alpha \frac{\partial J}{\partial w}, \quad b := b - \alpha \frac{\partial J}{\partial b}$$

Quá trình này được lặp lại nhiều lần (epochs), giúp các tham số dần hội tụ về giá trị tối ưu, làm giảm hàm mất mát và cải thiện độ chính xác của mô hình.

3.8. Confusion matrix

```
def confusion_matrix_binary(y_true, y_hat):
  true_positive = np.sum((y_true==1) & (y_hat==1))
  true_negative = np.sum((y_true==0) & (y_hat==0))
  false_positive = np.sum((y_true==0) & (y_hat==1))
```

```
    false_negative = np.sum((y_true==1) & (y_hat==0))

    return true_positive, true_negative, false_positive, false_negative
```

Ma trận nhầm lẫn cho biết mô hình nhầm lẫn ở dạng nào:

- **TP**: dự đoán đúng chữ số 1,
- **TN**: dự đoán đúng chữ số 0,
- **FP**: ảnh là 0 nhưng bị dự đoán thành 1,
- **FN**: ảnh là 1 nhưng bị dự đoán thành 0.

Những chỉ số này giúp phân tích sâu hơn chất lượng mô hình, thay vì chỉ nhìn vào accuracy.

3.8. Đánh giá mô hình Logistic Regression

Vì mỗi lần huấn luyện lại mô hình thì kết quả sẽ có sự sai lệch nên cần sử dụng hàm np.random.seed để cố định kết quả của mọi quá trình huấn luyện với cùng thông số huấn luyện.

```
def set_randomseed(seed=42):
    random.seed(seed)
    np.random.seed(seed)

    try:
        import tensorflow as tf
        tf.random.set_seed(seed)
    except:
        pass
```

Hàm đánh giá mô hình sẽ đánh giá và trả về các giá trị TP, TN, FP, FN sau khi đã truyền vào các tham số.

```
def evaluate(X, y, weights, bias, name="Test", save_path=None):
    y_prob = predict_probability(X, weights, bias)
    y_pred = predict(X, weights, bias)
    loss = loss_func(y, y_prob)
    accuracy = accuracy_func(y, y_pred)
    TP, TN, FP, FN = confusion_matrix_binary(y, y_pred)
```

Ví dụ đầu ra kết quả:

```
====== Test Evaluation ======
Loss     : 0.0091
Accuracy : 0.9991
```

TP = 1135
TN = 978
FP = 2
FN = 0